YARNNN: Autonomous Agent Platform for Recurring Knowledge Work

Canonical investor-facing architecture reference · March 2026

What YARNNN Is

YARNNN is an **autonomous agent platform** where persistent AI specialists do recurring knowledge work — and get smarter with every execution.

Each agent has its own identity, instructions, memory, data sources, schedule, and output history. Agents sleep between executions at zero cost. When they wake, they have full access to accumulated knowledge from their entire execution history, the user's synced work platforms, and learned preferences from past human corrections.

One sentence: Persistent AI agents that do your recurring work, learn from your feedback, and improve with tenure.

How It's Different

	Copilot Cowork	Claude Cowork	ChatGPT	YARNNN
Initiation	User hands off task	User starts session	User prompts	Scheduled, proactive, or event-driven — no human initiation required
Persistence	Task-scoped	Session-scoped	Session-scoped	Persistent agents that exist indefinitely
Memory	Work IQ / Microsoft Graph	Local filesystem	Conversation history	Per-agent + global user memory, compounding across executions
Learning	None	None	None	Feedback loop: user edits → learned preferences → better next output
Multi-agent	Single task	Single session	Single conversation	Many concurrent agents, each specializing in different work
Cross-platform	Microsoft only	Local machine only	None	Slack + Gmail + Notion + Calendar — synthesized
Idle cost	N/A	N/A	N/A	Zero — agents sleep between executions

The key insight: Session-based AI (Copilot Cowork, Claude Cowork, ChatGPT) starts from scratch every time. YARNNN agents compound. The 50th execution of a weekly status report is incomparably better than the 1st

— because the agent has 50 runs of accumulated memory, learned preferences, and retained knowledge.

Architecture Overview

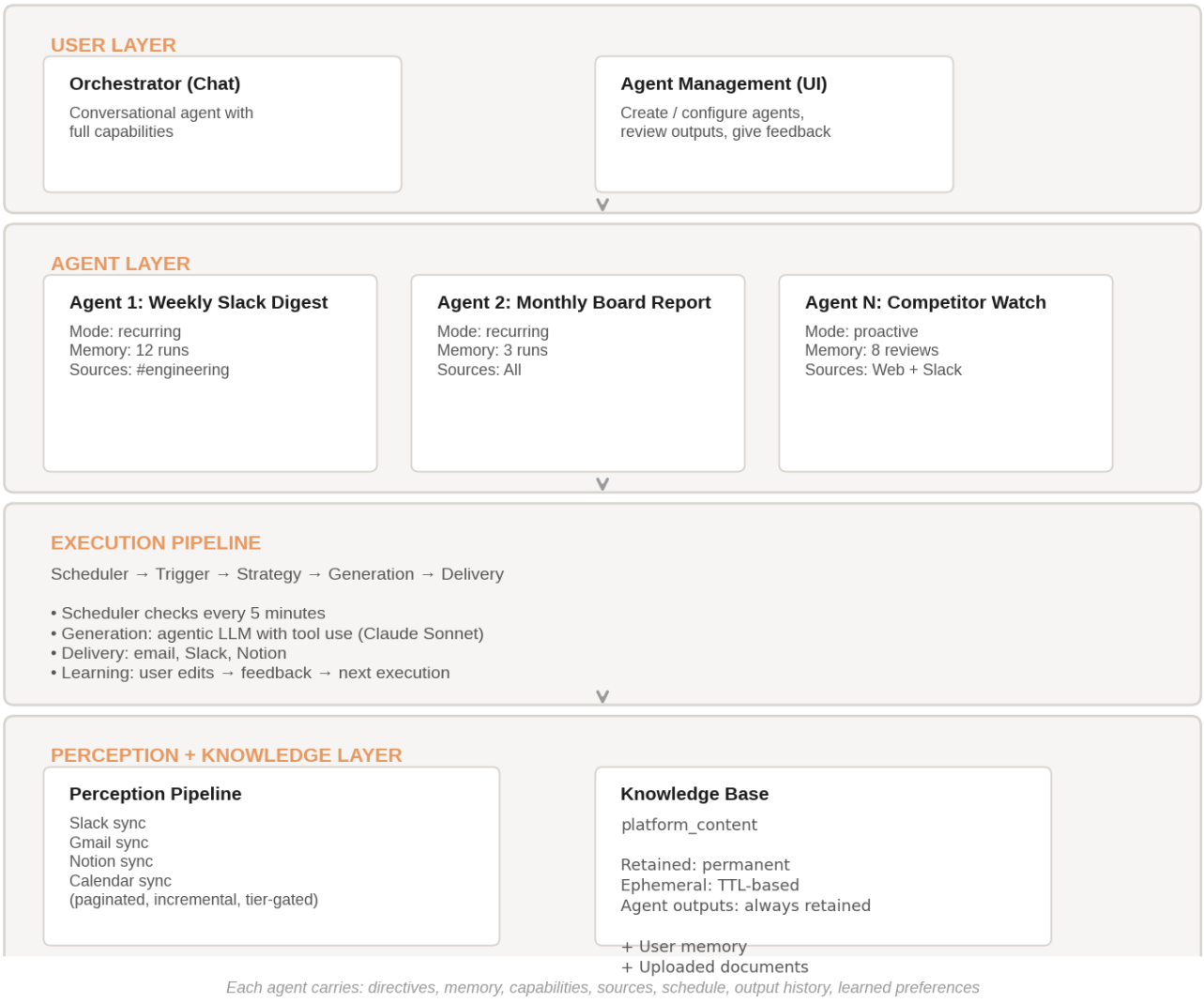

Figure 1: Platform architecture overview.

The Five Execution Modes

Every agent operates in one of five modes. The mode determines *when* the agent acts and *how* it decides.

Recurring — Clockwork Specialist

Fixed schedule. Always runs on time. The default for most knowledge work.

> *Example:* Weekly Slack digest, every Monday at 8am

Behavior: Wake → gather knowledge → generate → deliver → sleep

Goal — Project-Bound Specialist

Runs on schedule until an objective is met, then stops.

Example: Track product launch readiness, weekly until launch

Behavior: Wake → assess goal status → generate if still active → sleep (or complete)

Reactive — Event-Driven Specialist

Watches for platform events. Accumulates observations. Generates when a threshold is reached.

Example: Alert when 5+ urgent Slack threads pile up across channels

Behavior: Event arrives → note observation → threshold reached? → generate

Proactive — Living Specialist

Periodically reviews its domain. Decides autonomously whether to act, observe, or sleep longer.

Example: Competitive intelligence — scans sources, generates only when something significant emerges

Behavior: Review cycle → assess domain → generate / observe / sleep

Coordinator — Meta-Agent

Same as proactive, plus the ability to create new agents and direct existing ones.

Example: Watches your calendar and Slack; when it detects a big meeting coming, it spawns a meeting-prep agent

Behavior: Review cycle → assess domain → create child agent / advance another agent's schedule / observe / sleep

---

The Agent Intelligence Model

Every agent carries four layers of knowledge, assembled into its execution prompt:

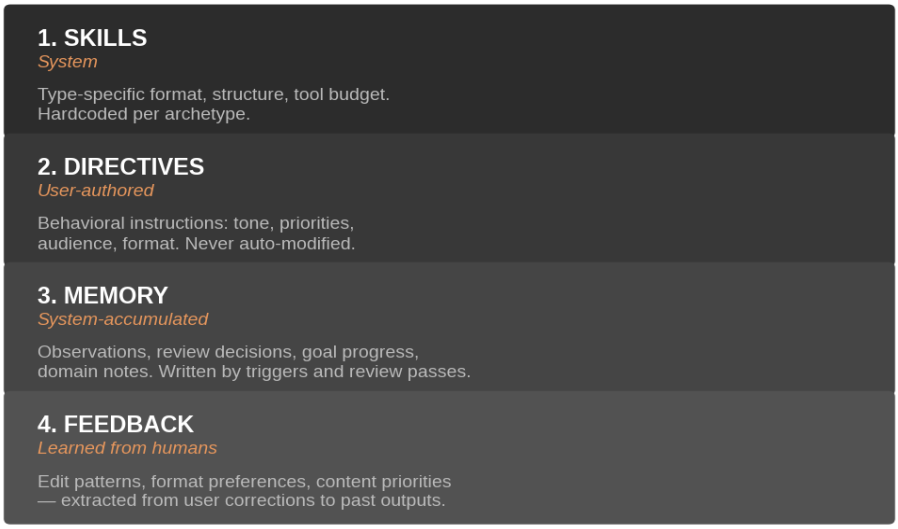

Figure 2: Four-layer agent intelligence model.

This is how an agent on its 50th run produces better output than its 1st: it has 50 executions worth of accumulated memory and learned preferences.

The Orchestrator

The Orchestrator is the user-facing conversational agent. It operates in real-time (streaming) with full capabilities:

- **Search** the knowledge base across all platforms
- **Create** new agents (configure, schedule, set directives)
- **Read** and explain agent outputs
- **Execute** platform actions (send Slack message, create calendar event)
- **Manage** agent configurations and memory

The Orchestrator knows the user's full context: connected platforms, active agents, recent conversations, accumulated memories. It's the control plane for the agent network.

When agents run autonomously (headless mode), they use a curated subset of read-only capabilities — search, read, list, web search. Write operations are reserved for coordinator agents (create/direct other agents) and the Orchestrator (user-supervised actions).

Knowledge Accumulation

The Perception Pipeline

YARNNN connects to four work platforms via OAuth:

Platform	What's synced	Retention
Slack	Channel messages, threads (expanded when substantive)	14 days
Gmail	Email threads by label (Inbox, Sent, Starred, Important)	30 days
Notion	Page content, tracked by last edit time	90 days
Calendar	Events, -7 days to +14 days	2 days

Sync is paginated, incremental (cursor-based), and tier-gated:

Free tier: Daily sync

Pro tier ($19/mo): Hourly sync

The Retention Model

Not all knowledge is equal. YARNNN retains what proved significant:

Ephemeral content (default): Synced with TTL. Slack messages expire after 14 days, Gmail after 30.

Retained content: When an agent references content during execution, or the Orchestrator references it in conversation, that content is marked *retained* — kept indefinitely.

Agent outputs: Everything an agent produces is written back to the knowledge base as permanently retained, searchable content. This closes the loop: one agent's output becomes another agent's input.

Over time, the knowledge base accumulates a curated corpus of everything that actually mattered in the user's work. This is the compounding advantage — a corpus that grows more valuable with every agent execution and every user interaction.

Infrastructure

Four services on Render (Singapore region):

Service	Role
API (Web Service)	FastAPI — chat, agent CRUD, auth, all user-facing endpoints
Agent Scheduler (Cron, every 5 min)	Triggers due agents, runs proactive review passes, nightly memory extraction
Perception Sync (Cron, every 5 min)	Platform sync for all connected users (tier-gated frequency)
MCP Server (Web Service)	Exposes YARNNN agents to Claude.ai and Claude Desktop via MCP protocol

No Redis, no queue, no background worker. All execution is inline. Agents sleep between cron ticks at zero compute cost.

LLM stack:

Agent generation + Orchestrator chat: Claude Sonnet 4 (Anthropic)

Proactive review passes: Claude Haiku 4.5 (cost-efficient lightweight reasoning)

Memory extraction: Claude Sonnet 4

Monetization

	Free	Pro ($19/mo)
Agents	2	10
Messages (Orchestrator)	50/month	Unlimited
Platform sources	5 Slack / 5 Gmail / 10 Notion	Unlimited
Sync frequency	Daily	Hourly
Platforms	All 4	All 4

Early Bird pricing: $9/month (Pro features, limited availability).

What’s Built and Working

Four-platform OAuth + landscape discovery + smart source auto-selection

Tier-gated perception pipeline (all 4 platforms, paginated, incremental)

Knowledge base with retention-based accumulation

Orchestrator agent — streaming, full capabilities, context-aware

Headless agent execution — agentic LLM loop with tool use

All 5 execution modes (recurring, goal, reactive, proactive, coordinator)

Per-agent directives + memory

Agent-scoped Orchestrator sessions (chat attached to specific agent)

Four-layer intelligence model (skills, directives, memory, feedback)

Email delivery via Resend

MCP server (OAuth 2.1 for Claude.ai, bearer token for Claude Desktop)

Nightly memory extraction from Orchestrator conversations

Agent outputs as searchable knowledge (accumulation loop closed)

2-tier monetization with Lemon Squeezy

The Thesis

The AI landscape is converging on agents as the work abstraction. Microsoft Copilot Cowork, Claude Cowork, and others are building session-based agents that execute tasks on demand.

YARNNN's bet is that **the most valuable work is recurring, not one-off** — and that recurring work benefits most from persistent agents with accumulated intelligence.

A session-based agent that writes your weekly status report starts from scratch every time. A YARNNN agent that has written 50 weekly status reports knows which metrics matter, which format your stakeholder prefers, which Slack channels have the signal vs. the noise, and which topics you always edit out.

That accumulated intelligence is the product. The agent is how it's delivered.

For implementation details, see Agent Execution Model. For the decision record behind this reframe, see ADR-103.